Arabian Journal for Science and Engineering
https://doi.org/10.1007/s13369-026-11720-1

RESEARCH ARTICLE-COMPUTER ENGINEERING AND COMPUTER SCIENCE

Attention-Augmented Lightweight 3D LiDAR Object Detection for Rubber Trees via MobilePIXOR and Model Distillation

Hoang Ngoc Tran[1] · Cong Thanh Nguyen[1] · Truong Thien Duong[1] · Minh Le Nguyen[1] · Nghi Vinh Nguyen[1]

Received: 11 September 2025 / Accepted: 27 September 2026
© King Fahd University of Petroleum & Minerals 2026

Abstract

This study presents a task-specific and deployment-oriented framework for lightweight rubber tree detection using 3D LiDAR point cloud data. The work focuses on the design, configuration, and embedded evaluation of an attention-enhanced BEV detector based on MobilePIXOR for rubber plantation perception under resource-constrained edge deployment. The proposed framework uses a geometry-only BEV occupancy representation and systematically investigates the integration of multi-scale attention and channel-spatial squeeze-and-excitation modules, including attention placement, reduction ratio selection, and fusion strategy design. At the standard $AP_{[.5,.95]}$ metric, the best-performing configuration, *MobilePixor + scse(8) + msa(3 + mul + BN)*, achieves an average precision of 0.717, outperforming the original MobilePIXOR, classical PIXOR, and representative non-PIXOR LiDAR baselines. To further reduce model complexity, knowledge distillation is applied to derive compact student models, achieving up to a 6.7× reduction in parameters while preserving competitive detection accuracy. The method is evaluated on a custom-built LiDAR dataset constructed for rubber tree plantation scenarios, with additional qualitative transferability assessment on real forest data. Overall, the results demonstrate that systematic attention configuration and distillation-based compression can improve the accuracy-complexity trade-off of MobilePIXOR for LiDAR-based tree detection under edge-oriented deployment constraints.

Keywords 3D point cloud · Rubber tree detection · MobilePixor · SCSE · MSA · Knowledge distillation

1 Introduction

Robotics is increasingly recognized as a driving force in the evolution of modern agriculture, offering innovative and scalable solutions to address persistent challenges such as labor shortages, inconsistent yields, and sustainability concerns [1]. Recent developments in autonomous systems and AI have paved the way for the deployment of intelligent field robots capable of executing agricultural tasks with high precision and reliability. Among high-value crops, rubber trees are particularly important due to their significant contribution to the global latex supply chain [2]. Nevertheless, rubber tree cultivation and harvesting continue to rely heavily on manual labor, particularly for tapping and maintenance, which limits scalability and increases operational costs [3]. These issues are compounded by rising global demand for natural rubber and growing climate volatility, particularly in Southeast Asia–where most of the world's rubber is produced and where environmental disruptions and labor shortages are increasingly common [4]. In this context, incorporating lightweight and autonomous robotic solutions into plantation operations is not only timely but essential to improve efficiency and support sustainable agro-industrial practices.

Despite its potential, deploying robotic systems in rubber plantations presents substantial technical challenges. The complex forest environment–with dense canopies, variable lighting, and frequent occlusion from overlapping vegetation–significantly impairs the performance of perception systems, especially those based on visual sensors [5]. Moreover, fluctuations in ambient lighting and seasonal

✉ Hoang Ngoc Tran
hoang2531992@gmail.com

Cong Thanh Nguyen
congnt.ce191449@gmail.com

Truong Thien Duong
truongdtce182403@fpt.edu.vn

Minh Le Nguyen
leminhhu.edu@gmail.com

Nghi Vinh Nguyen
nghinvce182108@fpt.edu.vn

[1] FPT University, Can Tho 94000, Vietnam

Published online: 12 October 2026

changes further hinder the reliability of vision-based detection. Compounding this, many high-performing object detection models demand substantial computational resources, making them impractical for deployment on low-power, embedded devices commonly used in field robotics [6, 7]. This discrepancy between algorithmic complexity and hardware constraints is a critical barrier to real-world adoption [8]. Therefore, achieving a balance between robust perception and computational efficiency is key to enabling practical autonomous operation in rubber plantations.

A number of studies have investigated deep learning-based detection strategies for rubber trees using various sensor modalities. For example, Wang et al. [9] applied Faster R-CNN [10] to perform detection by projecting LiDAR-based point cloud data onto 2D image planes, yielding favorable accuracy but requiring powerful computing resources. Similarly, Zhou et al. [11] utilized a mobile backpack-mounted LiDAR system for in-field data acquisition and employed PointPillars [12] and Swin Transformer [13] for object detection. Although these models achieved promising results, their high memory and computation demands render them unsuitable for embedded deployment in field robots.

This work focuses on the task-specific adaptation, systematic configuration, and deployment-oriented evaluation of an attention-enhanced MobilePIXOR framework for LiDAR-based rubber tree detection. The main contributions are summarized as follows:

- *Task-specific BEV LiDAR detector for rubber tree perception:* The lightweight MobilePIXOR architecture is adapted to rubber tree detection using a geometry-only BEV occupancy representation from 3D LiDAR point clouds. Oriented BEV boxes are predicted for tree localization while maintaining a compact model structure suitable for edge-oriented perception.
- *Systematic attention configuration analysis:* The integration of scSE and MSA modules into the MobilePIXOR framework is systematically investigated, including different scSE reduction ratios, MSA placements, and fusion strategies. This analysis clarifies how attention placement and fusion design affect detection accuracy, model complexity, and latency.
- *Knowledge-distillation-based student compression:* Knowledge distillation is applied to derive compact student models from the best-performing teacher configuration. The resulting student networks substantially reduce parameter count while preserving competitive detection accuracy.
- *Accuracy–complexity–latency evaluation:* The proposed framework is evaluated on a custom LiDAR dataset for rubber tree plantation scenarios and compared with PIXOR-based and representative non-PIXOR LiDAR baselines. The analysis includes detection performance, parameter count, FLOPs, and embedded inference latency on the Radxa Rock 5C/RK3588S2 platform.

The remainder of this paper is structured as follows. Section 2 reviews existing LiDAR-based object detection approaches relevant to forest and agricultural settings. Section 3 presents the proposed Attention-Enhanced MobilePixor architecture, detailing the full detection pipeline —from voxelization [14] to attention-based feature extraction and heatmap-based object localization. Section 4 reports the experimental results, comparing the proposed model with baseline methods and quantifying the impact of attention modules and knowledge distillation. Section 5 discusses model complexity, deployment considerations, limitations, and future work, and concludes the paper.

2 Related Work

The integration of autonomous robotic systems in agriculture has grown rapidly, addressing critical challenges such as labor shortages and environmental variability. Among sensing technologies, LiDAR stands out for its ability to provide accurate 3D spatial information, robustness to lighting changes, and large field of view [1]. These properties make it well-suited for deployment in forested environments like rubber plantations, where visual occlusion and lighting inconsistencies are common.

In this context, we utilize 3D LiDAR point clouds, where each point $p_i = (x_i, y_i, z_i)$ represents spatial coordinates in Euclidean space. The entire point cloud is denoted by $P = \{p_1, p_2, ..., p_n\}$. Unlike approaches that incorporate intensity or reflectance values, our system relies solely on geometric data–appropriate for mobile robots where vertical information is less critical for navigation and object discrimination.

The goal is to detect and localize m objects within P, each represented by:

- A class probability vector $c_j \in \mathbb{R}^k$,
- A confidence score $s_j \in [0, 1]$,
- An oriented BEV bounding box $b_j = (x_j, y_j, w_j, l_j, \theta_j) \in \mathbb{R}^5$. Here, w_j and l_j denote the ground-plane dimensions of the tree box, and the vertical height is treated as a fixed parameter rather than a regression target.

This can be formalized as a mapping:

$$f_{\text{det}} : P \rightarrow \left\{(c_j, s_j, b_j)\right\}_{j=1}^{m} \tag{1}$$

Earlier LiDAR-based detection methods relied on handcrafted features and exhaustive search [15], which lacked

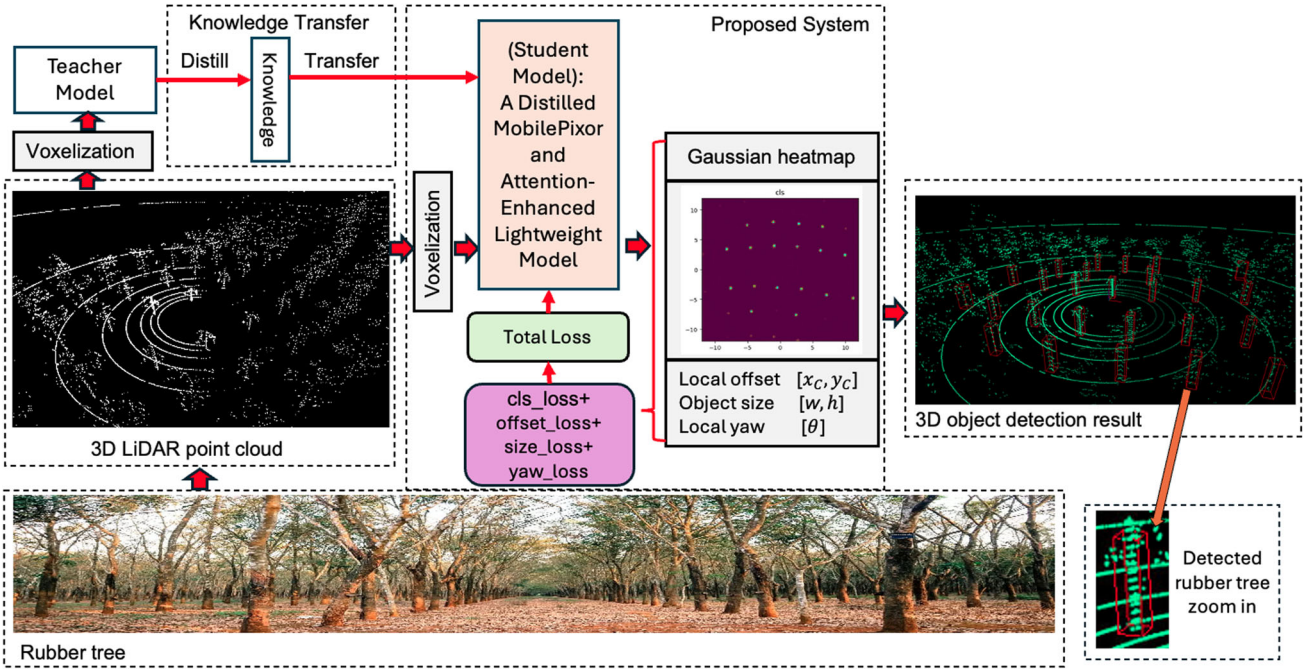


Fig. 1 Overview of the proposed rubber tree detection pipeline using the attention-enhanced MobilePixor model

scalability. Voxel-based deep learning approaches like VoxelNet [14] improved performance via 3D convolutions on voxelized grids, but incurred high computational costs. PointPillars [12] offered a compromise by converting point clouds into pseudo-images, enabling faster 2D convolutions (Fig. 1).

For real-time applications, PIXOR [7] adopted a BEV (bird's eye view) representation and a single-stage convolutional detector that directly regresses oriented bounding boxes. However, despite its speed, PIXOR's backbone remains too large for resource-constrained edge devices.

To address this, MobilePixor [16] was introduced. It replaces the backbone with a MobileNet encoder, incorporates depthwise separable convolutions, and uses Gaussian heatmap regression for object localization. This results in significantly fewer parameters and lower inference latency, while maintaining competitive accuracy. Its effectiveness has been demonstrated on the JRDB dataset [17].

Our work builds upon MobilePixor by systematically configuring and evaluating two attention mechanisms, scSE [18] and MSA, for LiDAR-based tree detection under cluttered plantation conditions. First, inspired by multi-scale attention mechanisms [19], *Multi-Scale Attention (MSA)* is applied along the U-Net skip connections, aggregating multi-kernel features (1×1, 3×3, 5×5) to improve spatial detail learning. Second, we embed *SCSE blocks* in each inverted residual block to recalibrate both spatial and channel-wise activations [18].

Algorithm 1 Voxelization of 3D LiDAR Point Cloud

Require: Point cloud $P = \{(x_i, y_i, z_i)\}_{i=1}^{n}$
Spatial resolution (v_x, v_y, v_z)
Bounds $[x_{\min}, x_{\max}]$, $[y_{\min}, y_{\max}]$, $[z_{\min}, z_{\max}]$
Ensure: Binary occupancy grid $O \in \{0, 1\}^{N_x \times N_y \times N_z}$
Compute grid sizes:

$$N_x \leftarrow \frac{x_{\max} - x_{\min}}{v_x}, N_y \leftarrow \frac{y_{\max} - y_{\min}}{v_y},$$

$$N_z \leftarrow \frac{z_{\max} - z_{\min}}{v_z}$$

Initialize $O(i, j, k) \leftarrow 0$ for all i, j, k
for all $p_i = (x_i, y_i, z_i) \in P$ **do**

$$v_x \leftarrow \left\lfloor \frac{x_i - x_{\min}}{v_x} \right\rfloor$$

$$v_y \leftarrow \left\lfloor \frac{y_i - y_{\min}}{v_y} \right\rfloor$$

$$v_z \leftarrow \left\lfloor \frac{z_i - z_{\min}}{v_z} \right\rfloor$$

$O(v_x, v_y, v_z) \leftarrow 1$
end for
return O

To further reduce model complexity, we employ knowledge distillation. A compact student network is trained to replicate a larger teacher model using a combination of hard targets, soft logits, and a feature similarity loss [20, 21], which improves convergence and representation alignment.

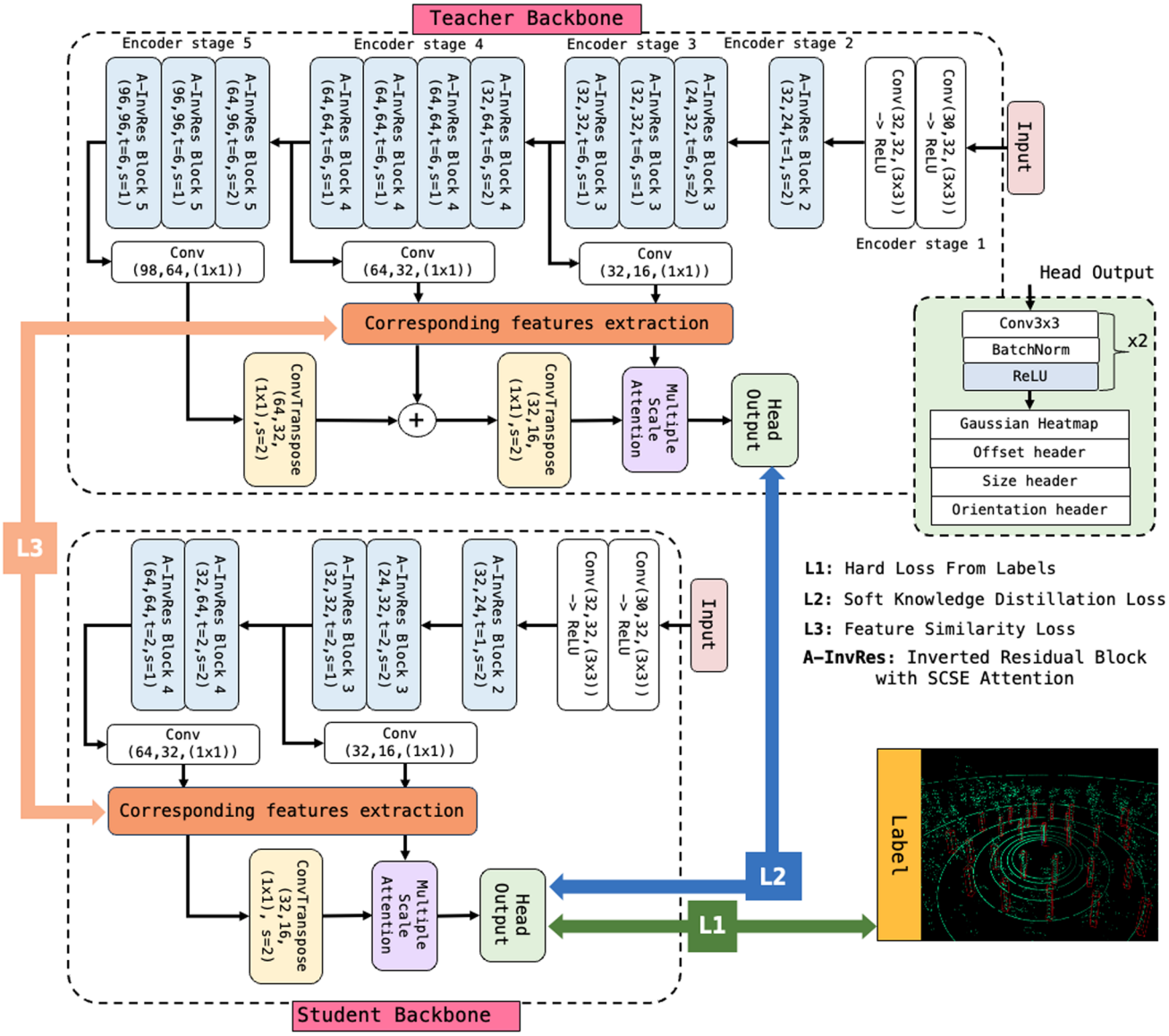


Fig. 2 Distillation framework overview: the proposed MobilePixor student backbone learns from both ground-truth labels (L1) and the teacher backbone via soft distillation loss (L2) and feature similarity loss (L3)

3 Proposed Method

3.1 Voxelization Process for LiDAR-Based 3D Object Detection

Voxelization is a key preprocessing step in 3D LiDAR object detection, transforming sparse and irregular point cloud data into a structured volumetric form suitable for deep learning. A point cloud $P = \{p_1, p_2, \ldots, p_n\}$ consists of points $p_i = (x_i, y_i, z_i)$ representing LiDAR measurements in continuous 3D space. The voxelization process discretizes this space into a uniform grid of volumetric cells, known as voxels, to provide a structured representation of the environment.

The voxel grid is defined by spatial resolution parameters along each axis and bounded by fixed minimum and maximum coordinates. The number of voxels per axis and the corresponding voxel index for each point are computed as outlined in Algorithm 1. A voxel is marked as occupied (value 1) if it contains at least one point, and unoccupied (value 0) otherwise. The resulting output is a sparse binary occupancy grid with dimensions $N_x \times N_y \times N_z$, preserving only relevant spatial regions and reducing computational overhead. Although the final detection objective is formulated on the BEV plane, the use of 3D LiDAR remains important because the vertical distribution of points provides useful structural cues for tree detection. The 30 height bins encode occupancy along the z-axis, helping the model distinguish tree

trunks from ground points, vegetation clutter, partial occlusions, and viewpoint-dependent noise. A 2D LiDAR may be sufficient for simplified trunk-diameter detection under controlled conditions, but it provides less information in uneven and cluttered plantation environments.

3.2 MobilePIXOR Architecture Enhanced with Attention Modules and Knowledge Distillation

3.2.1 Attention-Augmented Encoder Backbone for the Teacher Model

The proposed attention-augmented MobilePIXOR architecture is designed to efficiently process 3D LiDAR point cloud data for low-latency perception on embedded platforms. The model operates on voxelized input tensors of size $H \times W \times C$, where H, W, and C correspond to the height, width, and number of feature channels derived from the discretized 3D occupancy grid. As illustrated in Figure 2, both models share the same architectural design based on A-InvRes blocks, but differ in capacity. Three loss functions are used during training: the primary detection loss from labels (L1), a soft distillation loss on logits (L2), and a feature similarity loss (L3) between intermediate representations.

The teacher backbone begins with a shallow feature extraction stage consisting of two sequential 3×3 convolutions, each followed by batch normalization and ReLU activation. This stage projects the binary occupancy input into a 32-dimensional embedding space, yielding the low-level feature map C_1.

Subsequently, the network passes through four inverted residual stages (`A-InvRes Block2` to `A-InvRes Block5`). Each stage is defined by an expansion factor, number of output channels, repeat count, and stride. The first unit in each stage may downsample spatial resolution, while the remaining units preserve it. Within each inverted residual block, depthwise separable convolutions are employed to minimize parameter count and computational load. Skip connections are included when feasible to facilitate gradient flow and feature reuse. Additionally, each unit is enhanced with a concurrent spatial and channel squeeze-and-excitation (SCSE) block to recalibrate activations along both dimensions.

The decoder adopts a U-Net style top-down pathway. Feature maps from the last three encoder stages ($C_5 \in \mathbb{R}^{96\times H/16\times W/16}$, $C_4 \in \mathbb{R}^{64\times H/8\times W/8}$, and $C_3 \in \mathbb{R}^{32\times H/4\times W/4}$) are compressed to uniform channel sizes via 1×1 convolutions to enable lateral fusion. The 64-channel deep feature is upsampled and fused with the mid-level 32-channel map to form P_5, which is then upsampled again to match the spatial resolution of C_3, resulting in the decoder output P_4.

Depending on configuration, an optional Multi-Scale Attention (MSA) module is applied to refine spatial attention before final fusion with C_3. This progressive fusion strategy enables effective propagation of semantic context to higher-resolution maps, balancing detection precision and computational efficiency.

Details of the proposed attention blocks (A-InvRes), the integration of the MSA mechanism, and the knowledge distillation technique with the lightweight student backbone are presented in subsequent sections.

3.2.2 Attention Mechanisms

To enhance feature representation in both spatial and channel dimensions, we integrate two lightweight attention modules into the encoder-decoder architecture: the *Channel-Spatial Squeeze-and-Excitation (scSE)* block and the *Multi-Scale Attention (MSA)* module. These mechanisms are embedded at strategic layers to recalibrate feature responses and enrich context aggregation across skip connections. The structural designs of the proposed *A-InvRes* block with scSE, as well as the SCSE and MSA attention modules, are illustrated in Figure 3.

SCSE Attention: The scSE block jointly models channel-wise and spatial attention via two parallel branches:

– *Channel Squeeze-and-Excitation (CSE):* Global Average Pooling (GAP) compresses the spatial dimensions of each feature map to obtain a channel descriptor:

$$\mathbf{z} = \mathrm{GAP}(\mathbf{X}) \in \mathbb{R}^{B\times C}. \tag{2}$$

Two fully connected layers with reduction ratio r are then applied, where the first reduces the channel dimension from C to C/r, and the second restores it back to C:

$$\mathbf{h} = \mathrm{ReLU}(W_1\mathbf{z}), \quad W_1 \in \mathbb{R}^{\frac{C}{r}\times C}, \tag{3}$$

$$\mathbf{s} = \sigma(W_2\mathbf{h}), \quad W_2 \in \mathbb{R}^{C\times\frac{C}{r}}, \quad \mathbf{s} \in \mathbb{R}^{B\times C}. \tag{4}$$

The attention vector $\mathbf{s}$ is broadcast and multiplied with the original input:

$$\mathbf{X}_{\mathrm{cse}} = \mathbf{X} \odot \mathbf{s}. \tag{5}$$

– *Spatial Squeeze-and-Excitation (SSE):* A 1×1 convolution is applied to generate a spatial attention map, followed by sigmoid activation:

$$\beta = \sigma(\mathrm{Conv}_{1\times1}(\mathbf{X})) \in \mathbb{R}^{B\times1\times H\times W}, \tag{6}$$

$$\mathbf{X}_{\mathrm{sse}} = \mathbf{X} \odot \beta. \tag{7}$$

– *Fusion:* The outputs from CSE and SSE are concatenated along the channel axis and passed through another 1×1

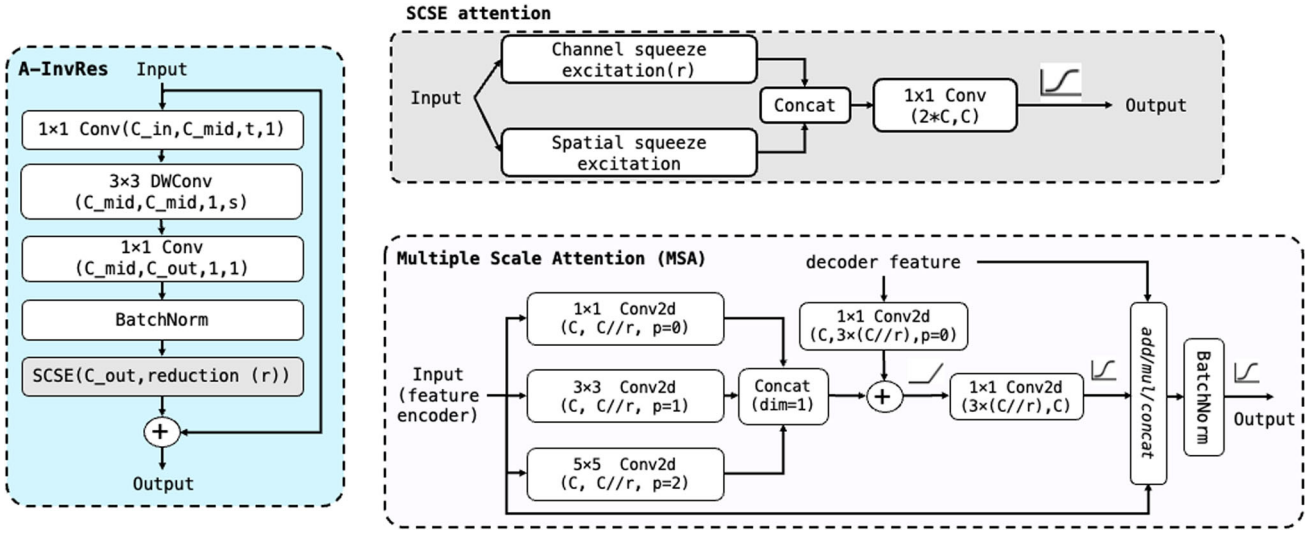


Fig. 3 Overview of the attention-augmented MobilePixor architecture with SCSE and MSA modules

convolution followed by sigmoid activation to produce the final attention map:

$$\mathbf{Y} = \sigma\left(\text{Conv}_{1\times1}\left([\mathbf{X}_{\text{cse}} \| \mathbf{X}_{\text{sse}}]\right)\right). \tag{8}$$

Multi-Scale Attention (MSA): The MSA module captures contextual information at different spatial scales by combining features extracted with multiple convolutional kernels. Given encoder feature $\mathbf{X}$ and decoder feature $\mathbf{D}$, the MSA proceeds in four stages:

– *Multi-kernel feature extraction:* Apply convolutions with kernel sizes 1×1, 3×3, and 5×5 to extract multi-scale features:

$$\theta_1 = \text{Conv}_{1\times1}(\mathbf{X}), \quad \theta_3 = \text{Conv}_{3\times3}(\mathbf{X}), \tag{9}$$

$$\theta_5 = \text{Conv}_{5\times5}(\mathbf{X}) \tag{10}$$

– *Concatenation:* The extracted features are concatenated:

$$\Theta = \text{Concat}(\theta_1, \theta_3, \theta_5) \tag{11}$$

– *Fusion with decoder:* Project the decoder feature map to match the dimension of Θ using:

$$\phi_g = \text{Conv}_{1\times1}(\mathbf{D}), \quad S = \Theta + \phi_g \tag{12}$$

– *Attention and output:* Pass the fused feature through a ReLU and a final 1×1 convolution followed by sigmoid to obtain attention mask ψ:

$$\psi = \sigma\left(\text{Conv}_{1\times1}(\text{ReLU}(S))\right) \tag{13}$$

Then, apply ψ to both encoder and decoder features and normalize:

$$\mathbf{F}_{\text{out}} = \sigma\left(\text{BN}(\psi \odot \mathbf{X} \odot \mathbf{D})\right) \tag{14}$$

Fusion Strategy and Optimization: Through ablation studies, we observed that introducing MSA at all skip connections does not necessarily yield proportional performance gains. A more effective strategy is to apply MSA selectively at the shallowest skip connection (encode stage 3), which provides a favorable balance between accuracy improvement and parameter efficiency.

To further refine the design, fusion variants combining scSE and MSA were investigated. Each variant is denoted as `scse(X) + msa(Y + Z + BN)`, where `X` corresponds to the reduction ratio r in the scSE module, `Y` specifies the encoder stage (using '&' for multiple stages) at which MSA is introduced, `Z` indicates the chosen fusion method (e.g., add, mul, or concat), and `BN` denotes the optional use of Batch Normalization after fusion. Among the tested configurations, the best-performing model was identified as `MobilePixor + scse(8) + msa(3 + mul + BN)`, in which multiplicative fusion with BatchNorm was applied. This configuration achieved the highest AP@[.5,.95] while preserving a compact footprint (716k parameters) and was therefore adopted as the teacher model for subsequent knowledge distillation. The detailed quantitative results of this design choice are presented and discussed in the experimental section.

3.3 Multi-Branch Detection Head

Following the design in [16], the detection head consists of four output branches that share an initial feature processing

stage composed of two sequential 3×3 convolutional layers, each followed by Batch Normalization and ReLU activation. These shared features are then fed into specialized prediction branches for object center probability, center offset, object size, and orientation.

The Gaussian Heatmap Header estimates the spatial probability distribution of object centers, producing an output tensor of size $\frac{H}{4} \times \frac{W}{4} \times C$, where H and W represent the input dimensions and C denotes the number of classes. For each coordinate (x, y), the value $p_i(x, y)$ reflects the probability that the center of an object belonging to class i is located there:

$$H_i(x, y) = \exp\left(-\frac{1}{2\sigma^2}\left[(x - x_c)^2 + (y - y_c)^2\right]\right) \tag{15}$$

Here, (x_c, y_c) is the true object center, and σ controls the Gaussian spread, typically scaled according to the object's size. This probabilistic representation softens the center encoding, mitigating size-related imbalance and improving detection robustness across scales.

The *Offset Header* regresses a continuous displacement vector $(\Delta u, \Delta v)$ to refine the coarse center positions derived from the heatmap, compensating for discretization introduced by feature map downsampling. The *Size Header* estimates the object's 2D dimensions (w, l), enabling accurate scale prediction for varying trunk diameters in plantation environments. It should be noted that the proposed detector does not regress the vertical height of the object. Although the input is a 3D LiDAR point cloud, the detection output is defined on the BEV plane as (x, y, w, l, θ). The height dimension is fixed in this study and is used only when rendering 3D boxes for visualization. Finally, the *Orientation Header* predicts the heading angle φ using a continuous $(\cos\varphi, \sin\varphi)$ representation, which avoids angular discontinuities and stabilizes training.

For training, the Gaussian heatmap branch is supervised with a modified focal loss as in [16], while the regression branches (offset, size, orientation) use L_1 loss with a binary mask to apply supervision only at valid object locations.

3.4 Unified Loss Formulation for Multi-Branch Detection Head

The overall training objective is formulated as the sum of individual loss components corresponding to all output branches in the detection head, encompassing both classification and regression tasks:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{offset}} + \mathcal{L}_{\text{size}} + \mathcal{L}_{\text{yaw}} \tag{16}$$

The classification branch, implemented via the Gaussian heatmap header, is supervised using a modified *focal loss* [22] to emphasize accurate object center predictions while suppressing the influence of extensive background regions. Formally, the classification loss $\mathcal{L}_{\text{cls}}$ is expressed as:

$$\mathcal{L}_{\text{cls}} = -\frac{1}{N}\sum_{x,y}\begin{cases}(1 - \hat{H}_{x,y})^{\alpha}\log(\hat{H}_{x,y}) \\ \quad \text{if } H_{x,y} = 1 \\ (1 - H_{x,y})^{\beta}\hat{H}_{x,y}^{\alpha}\log(1 - \hat{H}_{x,y}) \\ \quad \text{otherwise}\end{cases} \tag{17}$$

where $\hat{H}_{x,y}$ is the predicted heatmap value at spatial coordinate (x, y), and $H_{x,y} \in [0, 1]$ denotes the Gaussian-smoothed ground-truth heatmap. N is the number of valid object centers, with hyperparameters $\alpha = 2$ and $\beta = 4$. Although the detection task contains only one foreground class, the BEV heatmap is highly imbalanced because only a small number of grid cells correspond to rubber tree centers, whereas most cells belong to the background. Therefore, the use of focal loss is motivated by foreground-background imbalance in dense heatmap prediction rather than by multi-class imbalance. The Gaussian-smoothed target further reduces the penalty for locations near object centers, while the focal weighting suppresses easy background responses and emphasizes hard positive or near-positive regions. Binary cross-entropy with Gaussian weighting is also a reasonable alternative for this setting; however, we adopt the focal-style heatmap loss because it is consistent with the Gaussian heatmap detection head and provided stable convergence in our experiments.

The regression branches–responsible for predicting object center offset, 2D dimensions, and orientation–are optimized using the L_1 *loss*. A binary mask $M(u, v) \in \{0, 1\}$ is used to restrict loss computation to valid object locations. The regression loss for each branch is defined as:

$$\mathcal{L}_{\text{reg}} = \frac{1}{\sum M}\sum_{(u,v)} M(u, v) \cdot \left\| \hat{V}_{u,v} - V_{u,v} \right\|_1 \tag{18}$$

where $\hat{V}_{u,v}$ and $V_{u,v}$ denote the predicted and ground-truth vectors, respectively. This formulation applies to:

- $\mathcal{L}_{\text{offset}}$: regression of the continuous 2D center offset $(\Delta u, \Delta v)$;
- $\mathcal{L}_{\text{size}}$: regression of the object's width and length (w, l);
- $\mathcal{L}_{\text{yaw}}$: regression of the object's orientation via a continuous $(\cos\phi, \sin\phi)$ representation.

This unified formulation ensures consistent optimization across all prediction branches, facilitating stable convergence and improved multi-task learning performance (Fig. 4).

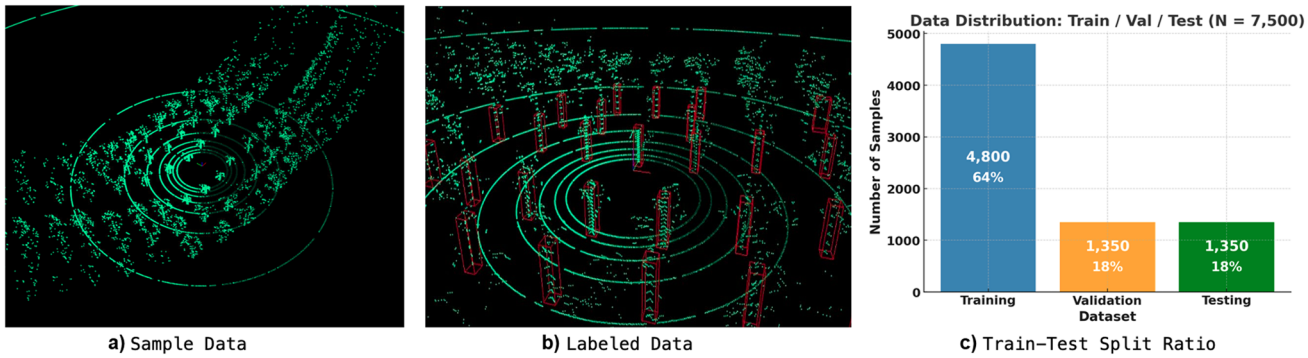


Fig. 4 Data preview

3.5 Model Compression via Knowledge Distillation and Pruning

The enhanced MobilePIXOR architecture, equipped with the proposed attention mechanisms and optimized through careful placement and configuration tuning, achieves strong detection performance and is designated as the teacher network for subsequent distillation. Although this architecture demonstrates promising accuracy for rubber tree detection, its relatively high computational cost poses challenges for real-time deployment on resource-constrained robotic platforms. To address this issue, a knowledge distillation strategy, combined with architectural pruning, is employed to derive a compact student network that retains the detection capability of the teacher model while substantially reducing computational complexity.

Pruning for Student Model Creation: The backbone network is constructed using *A-InvesBlock* units, defined as:

$$\text{A-InvesBlock}(in_c, out_c, t, s) \tag{19}$$

where in_c and out_c denote the input and output channel counts, t is the expansion factor, s indicates the spatial downsampling stride (applied only to the first sub-block), and n represents the number of repeated sub-blocks per stage.

The student models were derived from the best teacher configuration using different levels of structural reduction. Three representative model variants were evaluated, each with the final encoding stage removed (Block 5):

$$\begin{aligned} &\texttt{reduct_1}: \ (n = [1, 2, 2], \ t = [1, 2, 2]) \\ &\texttt{reduct_2}: \ (n = [1, 1, 2], \ t = [1, 1, 2]) \\ &\texttt{reduct_3}: \ (n = [1, 1, 2], \ t = [1, 2, 1]) \end{aligned} \tag{20}$$

where $n = [n_1, n_2, n_3]$ denotes the number of hidden blocks in each stage, with n_1, n_2, and n_3 corresponding to Stages 2, 3, and 4, respectively, and $t = [t_1, t_2, t_3]$ represents the expansion ratios for the corresponding stages.

Loss Functions for Student Network Training: The student network is trained under the joint supervision of ground-truth labels and the teacher network's outputs. Three complementary loss terms are employed (Fig. 3):

(L1) Hard Loss from Labels – This loss mirrors the teacher model's detection loss, enabling direct learning from annotated data:

$$\mathcal{L}_{\text{hard}} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{offset}} + \mathcal{L}_{\text{size}} + \mathcal{L}_{\text{yaw}} \tag{21}$$

Here, $\mathcal{L}_{\text{cls}}$ is a focal loss [22] applied to Gaussian heatmaps, while $\mathcal{L}_{\text{offset}}$, $\mathcal{L}_{\text{size}}$, and $\mathcal{L}_{\text{yaw}}$ are L_1 regression losses.

(L2) Soft Knowledge Distillation Loss – Following [20], the student is guided to match the teacher's softened output distributions via Kullback–Leibler (KL) divergence:

$$\text{soft}_k = \text{KL}\left(\log \frac{\exp(s_k/T)}{\sum_j \exp(s_j/T)} \,\middle\|\, \frac{\exp(t_k/T)}{\sum_j \exp(t_j/T)}\right) \tag{22}$$

where s_k and t_k denote the student and teacher logits for class k, T is the temperature, and $\text{KL}(p\|q)$ is the divergence from q to p. Using $T = 5$, the total soft loss is:

$$\mathcal{L}_{\text{soft}} = \text{soft}_{\text{cls}} + \text{soft}_{\text{offset}} + \text{soft}_{\text{size}} + \text{soft}_{\text{yaw}} \tag{23}$$

This loss is crucial for transferring the teacher's decision boundaries to the lightweight student.

(L3) Feature Similarity Loss – Inspired by [23], we enforce intermediate feature alignment between teacher and student backbones. Let $F_t^{(z)}$ and $F_s^{(z)}$ be feature maps from layer z of the teacher and student, respectively, chosen from `latlayer1` and `latlayer2` within Blocks 3 and 4. Cosine similarity is used:

$$\text{similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\max(\|x_1\|_2 \cdot \|x_2\|_2, \epsilon)} \tag{24}$$

where ϵ prevents division by zero. The feature similarity loss is:

$$\mathcal{L}_{\text{feat}} = \frac{1}{m} \sum_{z \in FS} \text{similarity}(F_s^{(z)}, F_t^{(z)}) \tag{25}$$

with m being the number of compared layers and FS the set of such layers.

Overall Training Objective: The final loss for student training is:

$$\mathcal{L}_{\text{total}} = (1 - \lambda_1 - \lambda_2)\mathcal{L}_{\text{hard}} + \lambda_1 \mathcal{L}_{\text{soft}} + \lambda_2 \mathcal{L}_{\text{feat}} \tag{26}$$

where λ_1 and λ_2 control the contribution of distillation and feature similarity terms. This joint optimization enables the compact student model to retain the teacher's predictive quality while supporting efficient model-level inference under edge deployment constraints.

4 Experiments

4.1 Dataset Preparation

Our custom 3D point cloud dataset, hosted on Hugging Face [24], comprises 5,000 frames generated in the Gazebo simulator to replicate a virtual rubber tree environment with natural variability in trunk inclination, height, canopy overlap, lighting, and partial occlusions, as shown in Fig. 5. Data were captured by a simulated robot equipped with a top-mounted LiDAR sensor. All synthetic frames were generated from the same mapped Gazebo plantation environment; therefore, the training, validation, and testing subsets were split from this single virtual environment rather than from separate simulated scenes. Each frame is stored as a *.bin point cloud file with corresponding annotations (object class, dimensions, position, and yaw). For the experiments, the original dataset of 5000 samples was first partitioned into training (2,300 samples), validation (1350 samples), and testing (1350 samples). Data augmentation was then applied exclusively to the training subset after this split, while the validation and testing subsets remained unchanged and non-augmented. The augmentation was performed with a 50% probability and included random rotation ($\pm 20°$), scaling (0.95–1.05), and translation (factor 0.4), thereby expanding the final training set to 4800 samples. This resulted in a total processed dataset of 7500 samples, including 4800 training samples, 1350 validation samples, and 1350 testing samples.

4.2 Network Configuration

The model processes BEV-encoded representations of 3D LiDAR point clouds, using a spatial resolution of $v_x = 0.05$, $v_y = 0.05$, and $v_z = 0.1$ meters. The region of interest covers $x \in [-12, 12]$, $y \in [-12, 12]$, $z \in [-1.5, 1.5]$ meters. After applying the encoding procedure described in Algorithm 1, The resulting input tensor has dimensions $480 \times 480 \times 30$, where the 30 channels correspond to vertical height bins along the z-axis. Each voxel stores a binary occupancy value indicating whether at least one LiDAR point falls within the corresponding 3D cell.

Training Configuration: Experiments are conducted on an Ubuntu 22.04 system with an Intel Xeon E5-2676 v3 CPU and an NVIDIA GeForce RTX 3060Ti GPU (8 GB). The model is implemented in PyTorch version 2.7.0 + cu12.8. To improve data loading throughput and fully utilize the GPU, the *prefetch_factor* and *num_workers* parameters in the *DataLoader* are optimized. Training is carried out for 56 epochs with a batch size of 4 for both training and validation.

Optimizer: The *Adam* optimizer is employed with adaptive learning rate adjustment. The initial learning rate is set to 2.2×10^{-4}, and a weight decay of 1.2×10^{-3} is applied at the training milestones (epochs 25 and 45).

Student Configuration: The student model configuration inherits from the teacher model setup, with key differences as follows: the learning rate is set to 2×10^{-4}, and the weight decay is reduced to 5×10^{-4}. Furthermore, $\lambda_1 = 0.29$ and $\lambda_2 = 0.41$ (26) yield the best performance for the student model, with a temperature of 5 applied in the *soft_loss* computation (Table 1).

4.3 Evaluation Metrics

To objectively evaluate the performance of different models, we combine multiple evaluation metrics to obtain a comprehensive assessment. The evaluation is conducted using two distance ranges: [0, 6] meters for predicting near objects and [6, 12] meters for predicting distant objects.

Intersection over Union (IoU) is used to match and rank predicted bounding boxes. The general formulation is given by:

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} \tag{27}$$

where IoU measures the spatial overlap between predicted boxes and ground-truth annotations. It is also used to remove duplicate predictions for the same object and as a threshold for matching predictions to ground-truth labels. In our experiments, we evaluate IoU thresholds of 0.7, 0.8, and 0.9.

Average Precision (AP) is employed as the primary metric for handling imbalanced datasets, as it is widely adopted in object detection benchmarks. AP is based on two fundamental measures: Precision and Recall, defined respectively as the ratio of true positive predictions to all predictions, and the ratio of true positive predictions to all ground-truth

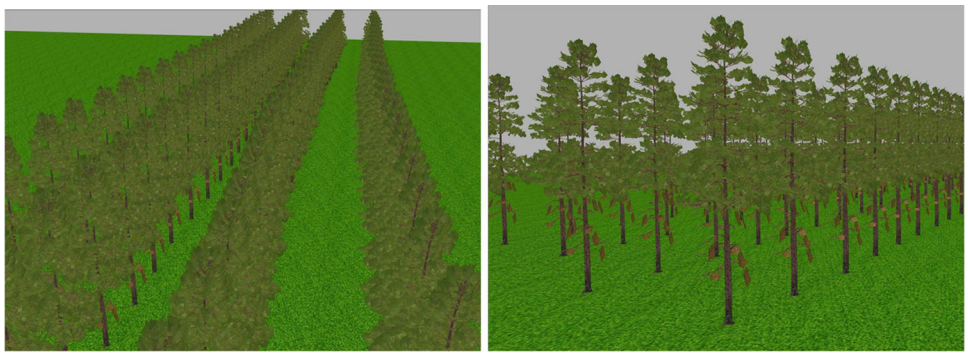


Fig. 5 Representative Gazebo-rendered views of the simulated rubber tree plantation environment

Table 1 Training and model configuration

Parameter	Value
Input encoding	
Voxel size (v_x, v_y, v_z)	(0.05, 0.05, 0.1) m
Region of Interest (ROI)	$x, y \in [-12, 12]$ m, $z \in [-1.5, 1.5]$ m
Input tensor size	$480 \times 480 \times 30$
Channels	Binary Occupancy Grid (30 Height Bins)
Training environment	
OS	Ubuntu 22.04
CPU	Intel Xeon E5-2676 v3
GPU	NVIDIA RTX 3060Ti (8 GB)
Framework	PyTorch 2.7.0 + cu12.8
DataLoader	Optimized `prefetch_factor`, `num_workers`
Training configuration (teacher model)	
Epochs	56
Batch size	4 (train/val)
Optimizer	Adam
Initial learning rate	2.2×10^{-4}
Weight decay	1.2×10^{-3} (milestones: 25, 45)
Student model configuration	
Learning rate	2.0×10^{-4}
Weight decay	5.0×10^{-4}
λ_1	0.29
λ_2	0.41
Temperature (soft loss)	5

instances:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{28}$$

where TP denotes true positives, FP false positives (Type I errors), and FN false negatives (Type II errors). In addition, *Accuracy* is used as a secondary measure to determine the overall correctness of predictions:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} \tag{29}$$

where TN represents true negatives.

AP is computed as the area under the Precision–Recall curve, obtained by integrating Precision with respect to Recall over the interval [0, 1]. A larger area corresponds to a higher AP, indicating better model performance:

$$\text{AP} = \int_0^1 \text{Precision}(\text{Recall})\, d(\text{Recall}) \tag{30}$$

The AP of a special IoU is the average of range [0,6] and [6,12]

$$\text{AP}_t = \frac{1}{2}(\text{AP}_t^{[0,6]} + \text{AP}_t^{[6,12]}) \tag{31}$$

Average AP (AP@[.5,.95]), introduced in [25], provides a standardized benchmark for model comparison. AP is first computed at individual IoU thresholds ranging from 0.5 to 0.95 in increments of 0.05. The final AP@[.5,.95] is obtained by averaging these values:

$$\text{AP@[.5,.95]} = \frac{1}{10} \sum_{t=0}^{9} \text{AP}_{0.5+0.05t} \tag{32}$$

This aggregated metric serves as the basis for objective comparison across models with different configurations.

5 Results and Discussion

The detection performance of the proposed model on rubber tree instances is presented in Fig. 6. For objects within the range [0,6], the model consistently achieves high-confidence predictions with strong localization accuracy. In the interval [6,12], the detection accuracy decreases slightly; however, the majority of predictions remain within an acceptable tolerance. Notably, for application scenarios that do not impose

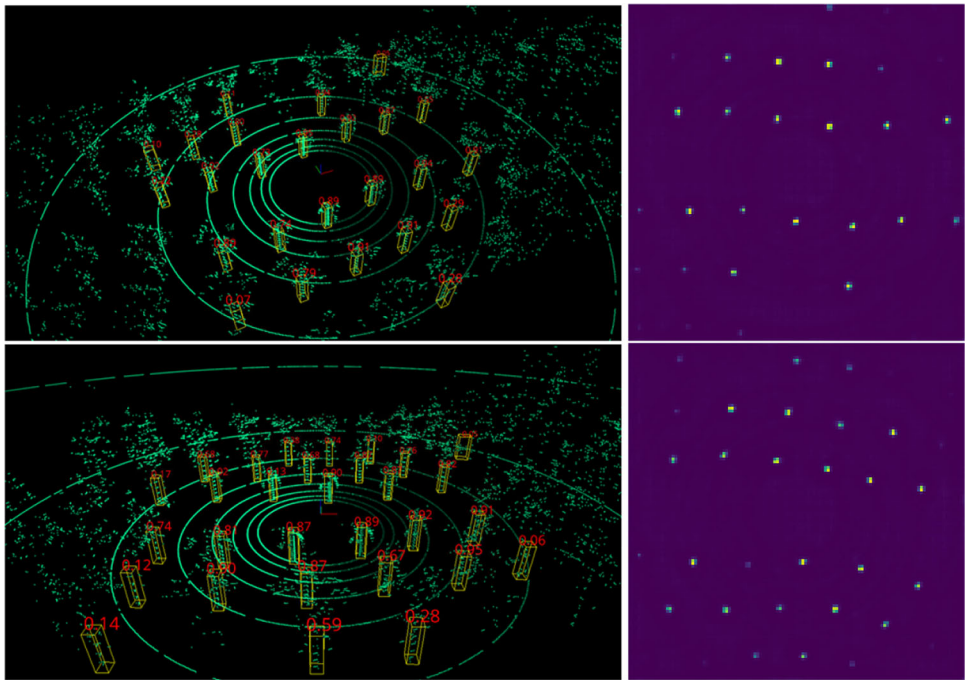


Fig. 6 Rubber tree detection results and corresponding heatmap representations

stringent IoU thresholds, the model yields competitive and reliable results across different distance ranges, as reported in Table 2.

5.1 Object Detection Results

Teacher results: Table 2 summarizes the detection performance of various teacher backbone architectures. The baseline MobilePixor model, without any architectural enhancements, contains 596K parameters and yields an average AP@[.5,.95] of 0.678. When attention mechanisms are introduced individually, noticeable improvements are observed. Incorporating scSE blocks enhances channel–spatial feature calibration and increases AP@[.5,.95] to the range of 0.697–0.706, albeit at the cost of slightly higher latency. Similarly, introducing MSA modules improves multi-scale spatial representation, achieving AP@[.5,.95] values around 0.689–0.694, while maintaining relatively low latency.

More substantial gains are obtained when combining both mechanisms. The integration of scSE and MSA consistently outperforms individual attention variants, with AP@[.5,.95] rising to 0.702–0.715 across different configurations. Among these, the optimal design, MobilePixor + scse(8) + msa(3 + mul + BN), achieves the highest average AP@[.5,.95] of 0.717. This corresponds to a relative improvement of 3.9% compared to the baseline MobilePixor, while maintaining a compact model size (716K parameters). Importantly, this configuration consistently delivers superior accuracy across all IoU thresholds (0.7, 0.8, and 0.9), establishing it as the most effective teacher backbone for subsequent knowledge distillation. Furthermore, it surpasses the classical Pixor by 12.7% in average AP@[.5,.95], highlighting the advantage of the proposed lightweight attention-augmented design.

In addition to PIXOR-based baselines, two representative non-PIXOR LiDAR detectors, VoxelNet and PointPillars, were also included for comparison. As shown in Table 2, VoxelNet obtains $AP_{@[.5,.95]} = 0.629$ with 6.43M parameters and 134.51 GFLOPs, while PointPillars obtains $AP_{@[.5,.95]} = 0.625$ with 4.82M parameters and 17.03 GFLOPs. In comparison, the proposed *MobilePixor + scse(8) + msa(3 + mul + BN)* achieves a higher $AP_{@[.5,.95]}$ of 0.717 with only 716K parameters and 7.04 GFLOPs. These results indicate that the proposed attention-augmented MobilePixor provides a more favorable accuracy-complexity trade-off than the evaluated non-PIXOR baselines under the same dataset and evaluation protocol.

Student results: The student models were derived from the best teacher configuration (MobilePixor + scse(8)+ msa(3 + mul + BN)) through varying levels of structural reduction. Among these, the *Student_MobilePixor + scse(8) + msa(3 + mul + BN)_reduct_3* variant attains competitive performance with only 89K parameters, corresponding to a 6.7× reduction in model size while preserving accuracy comparable to the original baseline. In contrast, the *Student_MobilePixor + scse(8) + msa(3 + mul + BN)_reduct_1* variant provides the most favorable balance between accuracy and compactness, achieving a 1.7% improvement in detection performance with just 115K parameters (5.2× reduction).

Table 2 Performance comparison of teacher backbone networks and non-PIXOR LiDAR baselines under different IoU thresholds.

Teacher backbone network	Params	FLOPs (G)	Latency (ms)	IoU 0.7		IoU 0.8		IoU 0.9		AP@[.5,.95]
				[0,6]	[6,12]	[0,6]	[6,12]	[0,6]	[6,12]	
VoxelNet	6.43M	134.51	–	0.862	0.891	0.477	0.581	0.058	0.075	0.629
PointPillars	4.82M	17.03	–	0.827	0.913	0.399	0.598	0.049	0.087	0.625
Pixor	2.46M	53.58	179.25 ± 5.43	0.883	0.885	0.471	0.602	0.048	0.092	0.636
MobilePixor	596K	6.55	111.46± 5.95	0.952	0.924	0.590	0.720	0.075	0.132	0.678
MobilePixor + scse(8)	704K	6.88	127.08 ± 6.42	0.942	0.906	0.698	0.770	0.127	0.179	0.697
MobilePixor + scse(16)	698K	6.88	128.77 ± 8.06	0.967	0.922	0.704	0.788	0.126	0.181	0.706
MobilePixor + msa(3&4 + add)	649K	6.86	114.29 ± 6.63	0.940	0.922	0.690	0.745	0.115	0.137	0.689
MobilePixor + msa(3 + add)	607K	6.70	113.85 ± 7.14	0.952	0.926	0.664	0.761	0.116	0.154	0.694
MobilePixor + msa(3 + mul + BN)	608K	6.71	115.21 ± 6.03	0.948	0.939	0.657	0.731	0.096	0.143	0.700
MobilePixor + scse(8) + msa(3 + add)	715K	7.04	132.12 ± 9.86	0.945	0.925	0.660	0.770	0.125	0.175	0.702
MobilePixor + scse(16) + msa(3 + add)	709K	7.04	131.23 ± 8.62	0.938	0.917	0.682	0.755	0.113	0.171	0.693
MobilePixor + scse(8) + msa(3&4 + add)	757K	7.19	130.61 ± 7.80	0.939	0.910	0.639	0.756	0.088	0.151	0.684
MobilePixor + scse(16) + msa(3&4 + add)	751K	7.19	130.76 ± 7.67	0.953	0.917	0.696	0.748	0.114	0.159	0.697
MobilePixor + scse(16) + msa(3&4 + add + BN)	755K	7.19	132.09 ± 8.14	0.940	0.933	0.668	0.806	0.131	0.178	0.705
MobilePixor + scse(16) + msa(3&4 + concat + BN)	755K	7.21	131.37 ± 13.00	0.963	0.935	0.704	0.806	0.137	0.203	0.715
MobilePixor + scse(16) + msa(3&4 + mul + BN)	755K	7.19	131.71 ± 10.03	0.953	0.924	0.718	0.809	0.123	0.196	0.712
MobilePixor + scse(8) + msa(3 + add + BN)	716K	7.04	132.50 ± 9.78	0.956	0.924	0.690	0.797	0.109	0.189	0.705
MobilePixor + scse(8) + msa(3 + concat + BN)	716K	7.05	129.52 ± 8.33	0.954	0.925	0.704	0.804	0.131	0.203	0.709
MobilePixor + scse(8) + msa(3 + mul + BN)	716K	7.04	128.97± 7.62	0.954	0.932	0.727	0.825	0.147	0.218	**0.717**

scse(X) + msa(Y + Z + BN), where X is the scSE reduction ratio r, Y is the encoder stage (using '&' for multiple stages) for MSA, Z is the fusion method (add/mul/concat, *default: add*) in MSA module, and BN indicates optional Batch Normalization

Table 3 Performance of student backbone networks after knowledge distillation.

Student Backbone Network	Params	FLOPs	Latency (ms)	IoU 0.7		IoU 0.8		IoU 0.9		AP@[.5,.95]
				[0,6]	[6,12]	[0,6]	[6,12]	[0,6]	[6,12]	
Student_MobilePixor + scse(8)_reduct_1	103K	5.09	107.80 ± 5.11	0.932	0.903	0.602	0.740	0.094	0.141	0.678
Student_MobilePixor + scse(8) + msa(3 + mul + BN)_reduct_1	115K	5.24	109.90 ± 6.61	0.946	0.919	0.637	0.760	0.101	0.162	**0.695**
Student_MobilePixor + scse(8) + msa(3 + mul + BN)_reduct_2	105K	5.04	105.16 ± 4.84	0.924	0.915	0.597	0.751	0.090	0.153	0.686
Student_MobilePixor + msa(3 + mul + BN)_reduct_3	67K	4.89	102.82 ± 3.92	0.819	0.862	0.372	0.451	0.042	0.053	0.602
Student_MobilePixor + scse(8) + msa(3 + mul + BN)_reduct_3	89K	5.05	104.78 ± 4.37	0.948	0.914	0.619	0.733	0.073	0.145	0.684

Real-World Detection Results: To further assess the practical applicability of the proposed approach, additional real-world forest data from the FinnWoodlands dataset [26] were used for qualitative inference. FinnWoodlands provides real forest-scene data, including RGB stereo images, point clouds, sparse depth maps, and manual annotations for forest objects such as tree trunks. Several representative frames were selected to visually examine the transferability of the trained detector to real forest-like environments, as shown in Fig. 10.

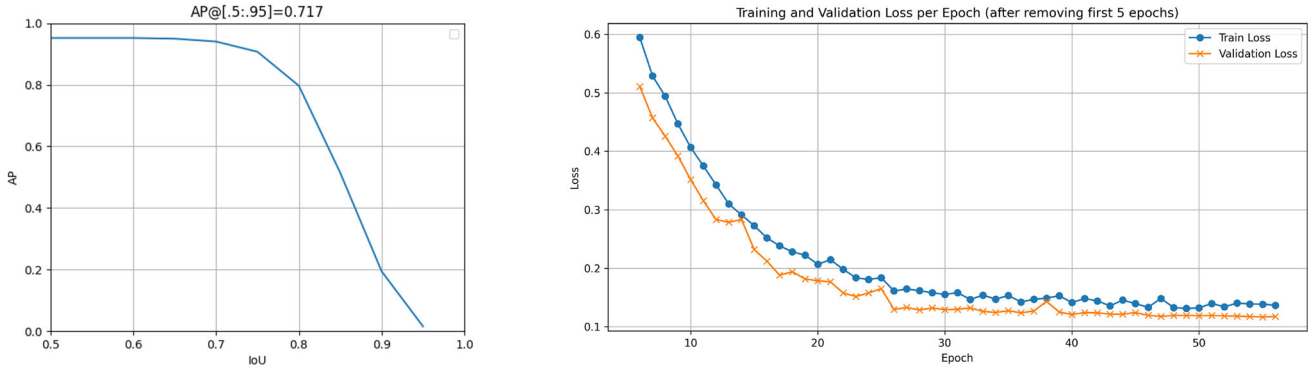
The selected point cloud data were processed using both the baseline MobilePixor model and the proposed MobilePixor + scse(8) + msa(3 + mul + BN) model. The corresponding detection results are presented in Fig. 10. In the visualization, red bounding boxes indicate detections produced only by the baseline MobilePixor model, while blue bounding boxes indicate detections produced only by the proposed model.

As observed in the bird's-eye-view (BEV) results, several tree-like objects located near the sensor were detected by the proposed model but missed by the baseline MobilePixor model. Conversely, at greater distances from the sensor, some objects were detected by the baseline model but not by the proposed model. These observations suggest that the proposed MobilePixor + scse(8) + msa(3 + mul + BN) model may improve the qualitative detection of nearby tree-like structures in real forest scenes. It should be noted that FinnWoodlands contains general forest scenes rather than rubber plantations. Thus, this experiment serves only as a qualitative transferability test, while comprehensive quantitative evaluation on annotated real rubber plantation datasets remains future work.

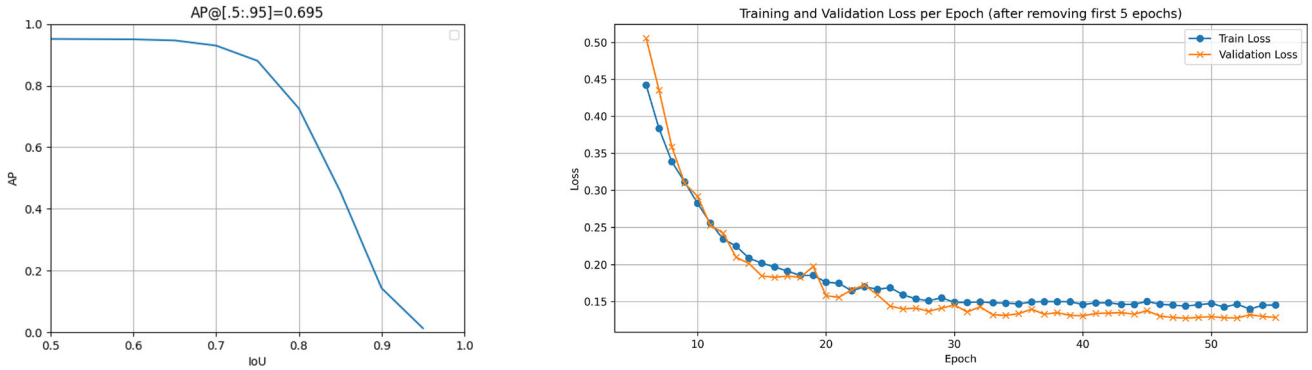
5.2 Result Discussion

In general, most models achieve their highest performance at lower IoU thresholds (≤ 0.7). Therefore, an IoU threshold of 0.7 or higher is employed as the standard reference for model comparison. At higher thresholds, such as IoU = 0.9, the stricter localization requirements substantially reduce detection accuracy, with the best-performing model reaching only 0.218. In contrast, the IoU = 0.8 setting serves as a more informative benchmark, as it highlights clear performance differences before and after the proposed architectural enhancements.

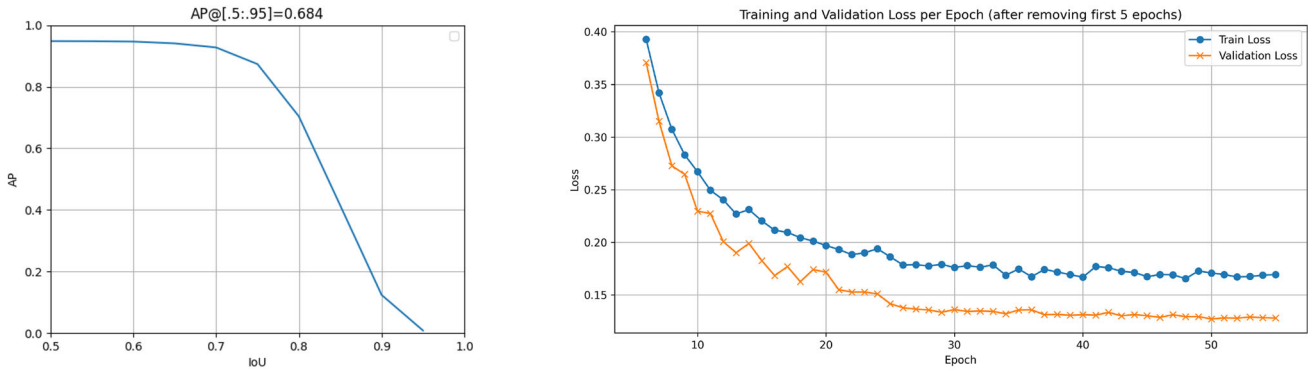
Teacher Training Discussion. The teacher backbone was developed via a systematic exploration of architectural variants, including independent evaluation of individual modifications, controlled combination experiments, and iterative refinements to ensure consistency. For the scSE module, multiple cSE-sSE combinations and reduction ratios were tested; concatenation with reduction ratios of 8 and 16 yielded the most accurate results and was therefore selected for subse-



(a) Model Teacher `MobilePixor+scse(8)+msa(3+mul+BN)` training result



(b) Model `Student_MobilePixor+scse(8)+msa(3+mul+BN)_reduct_1` training result



(c) Model `Student_MobilePixor+scse(8)+msa(3+mul+BN)_reduct_3` training result

Fig. 7 Comparison of model training results for different configurations. Each main subfigure shows the average AP plot (left) and training loss (right)

quent integration with MSA. For the MSA module, different placements were examined, and applying MSA to shallower lateral layers proved most effective. Finally, multiple scSE-MSA configurations were trained and compared, and the best-performing setup was adopted as the teacher backbone. To further isolate the contribution of the scSE module, an additional no-scSE ablation was conducted by evaluating *MobilePixor + msa(3 + mul + BN)* under the same training and evaluation protocol. As shown in Table 2, this no-scSE variant achieves $AP_{@[.5,.95]} = 0.700$ with 608K parameters, 6.71 GFLOPs, and 115.21 ms latency. In comparison, the best scSE-based configuration, *MobilePixor + scse(8) + msa(3 + mul + BN)*, achieves $AP_{@[.5,.95]} = 0.717$ with 716K parameters, 7.04 GFLOPs, and 128.97 ms latency. This comparison shows that adding scSE improves AP by 0.017, but increases latency by 13.76 ms on the tested embedded platform. Therefore, scSE should not be interpreted as latency-free. Instead, it acts as an accuracy-oriented feature recalibration module

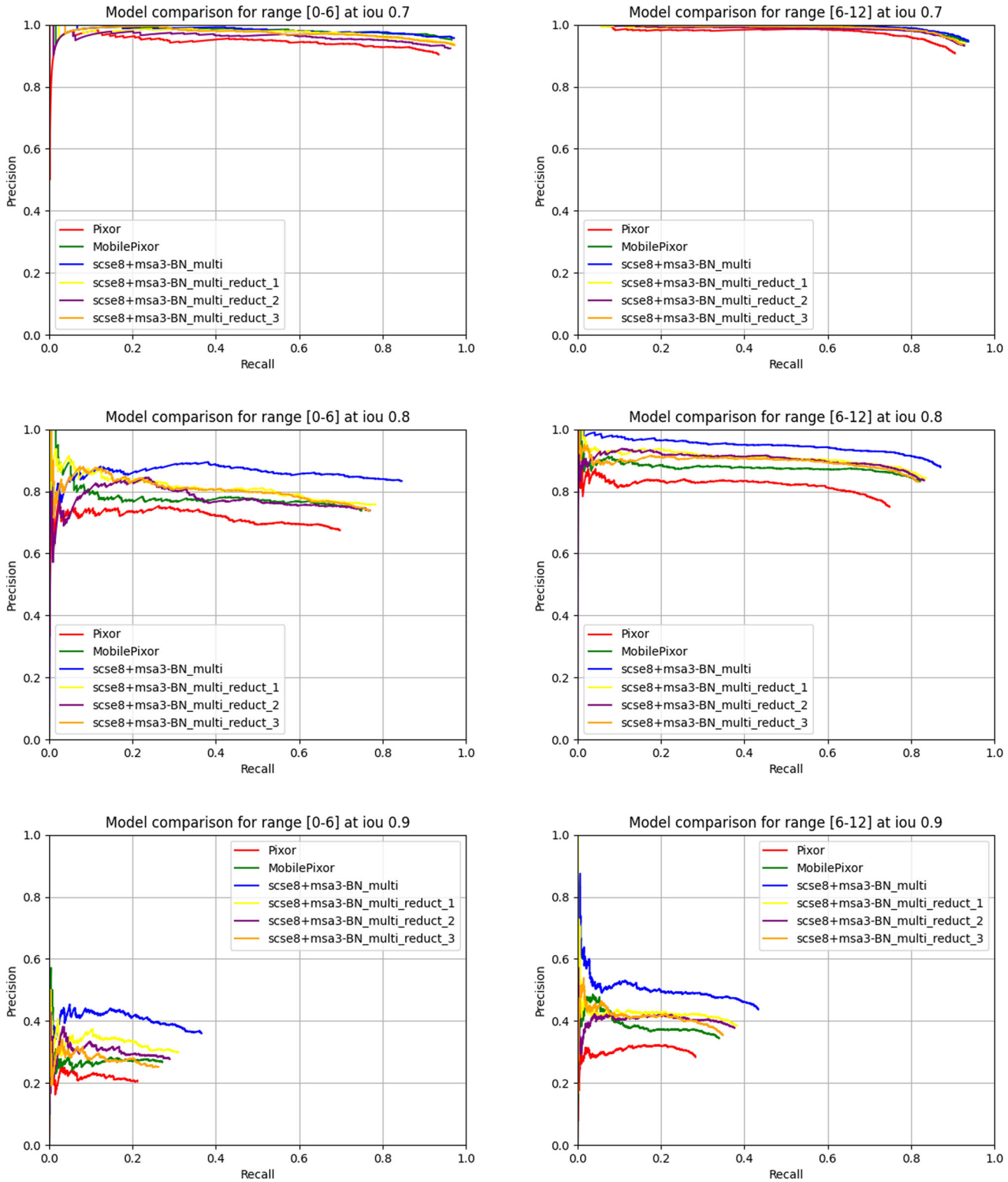
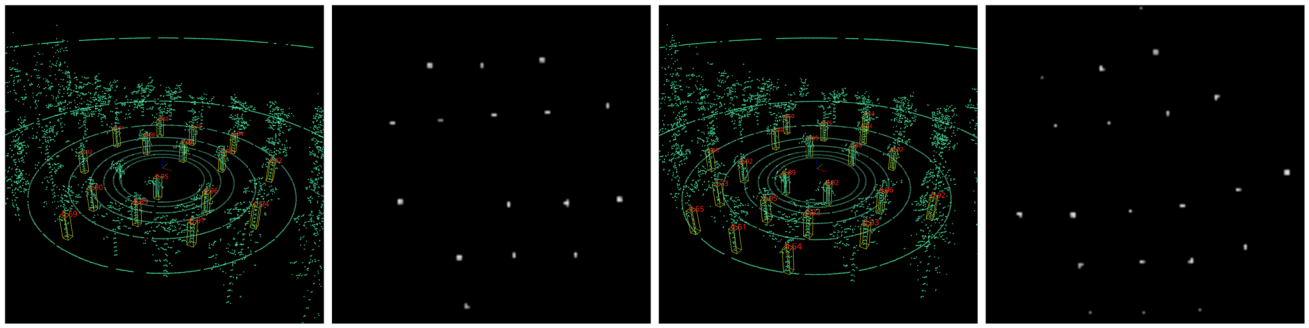
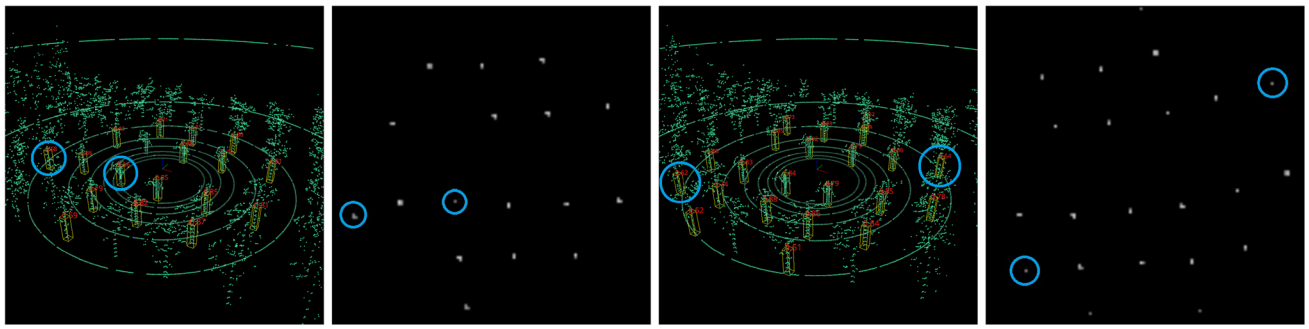


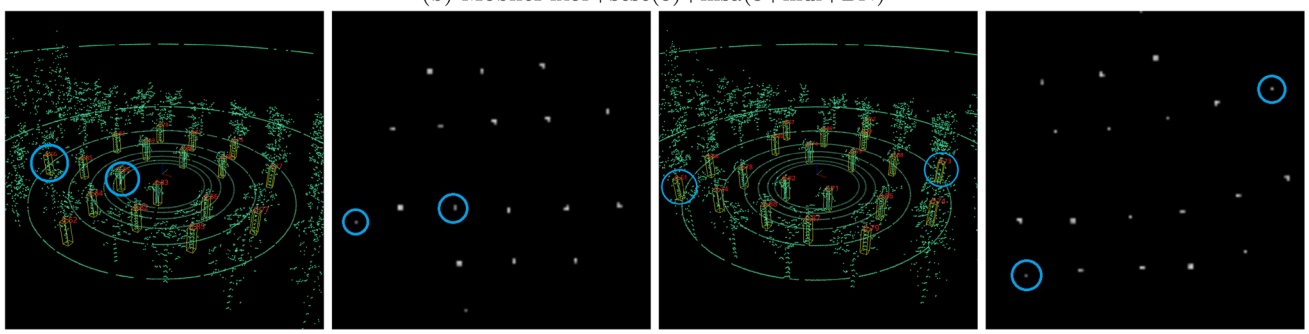
Fig. 8 Precision–Recall curves of teacher and different student models at IoU thresholds 0.7, 0.8, and 0.9



(a) MobilePixor



(b) MobilePixor+scse(8)+msa(3+mul+BN)



(c) Student_MobilePixor+scse(8)+msa(3+mul+BN)_reduct_1

Fig. 9 Rubber tree detection results. Each pair shows the point-cloud visualization with BEV-oriented boxes extruded using a fixed height for display purposes only (left), and the corresponding heatmap (right)

that improves detection performance, especially at stricter IoU thresholds, while introducing a non-negligible runtime overhead (Fig. 9).

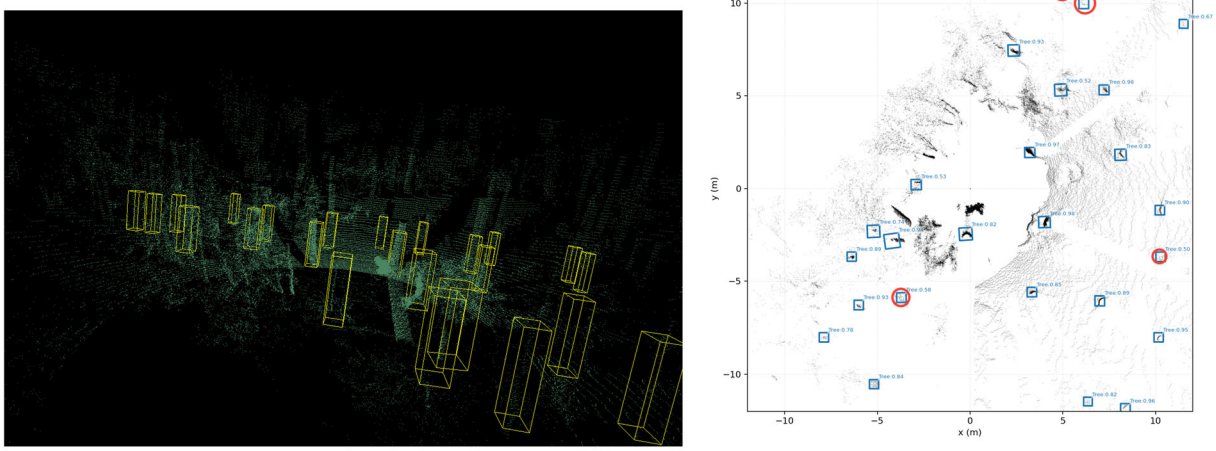
Student Training Discussion. The student networks inherit the architectural improvements of the teacher model, with complexity reduced mainly by shrinking encoder blocks. As shown in Table 3, this yields a clear trade-off between accuracy and efficiency: lighter variants significantly reduce parameters but lose some precision, while larger variants maintain higher accuracy at the cost of increased size.

A similar scSE-related trade-off is also observed in the student models. The no-scSE student variant, *Student MobilePixor + msa(3 + mul + BN) reduct 3*, obtains $AP_{@[.5,.95]} = 0.602$ with 67K parameters and 102.82 ms latency. By contrast, the corresponding scSE-based variant, *Student MobilePixor + scse(8) + msa(3 + mul + BN) reduct 3*, achieves $AP_{@[.5,.95]} = 0.684$ with 89K parameters and 104.78 ms latency. Although removing scSE slightly reduces latency and parameter count, it causes a substantial decrease in AP. This indicates that scSE is particularly useful for preserving detection accuracy under aggressive model compression.

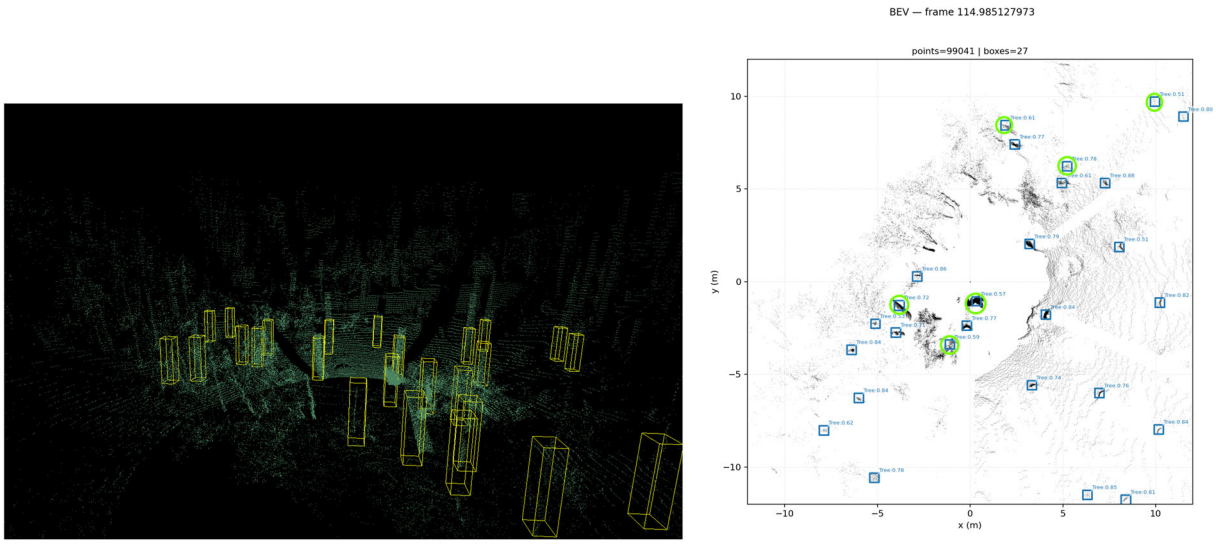
Two representative cases, `reduct_1` and `reduct_3`, are illustrated in Fig. 7 and Fig. 8. The compact `reduct_3` (89K parameters) shows stable but shallow convergence, with validation loss remaining above 0.15. In contrast, `reduct_1` (115K parameters) achieves deeper convergence, validation loss around 0.1, and consistently higher AP, offering the best balance between accuracy and compactness.



(a) Lidar projection onto RGB



(b) MobilePixor



(c) MobilePixor+scse(8)+msa(3+mul+BN)

Fig. 10 Qualitative real-forest detection results on the FinnWoodlands dataset [26]. The left image shows point-cloud visualizations with BEV-oriented detections rendered using a fixed height, while the right image shows the corresponding BEV view. The rendered 3D boxes are for visualization only and do not indicate height regression by the network

5.3 Model Complexity Analysis

The latency measurements were obtained on the *Radxa Rock 5C* platform, equipped with a 6-TOPs NPU based on the Rockchip RK3588S2. Each model was first converted to the ONNX format and subsequently to the RKNN format for execution on the device.

Model Size Analysis. Model size provides a practical indicator of computational complexity. The larger teacher models, namely `MobilePixor + scse(16) + msa(3&4 + concat + BN)`, `MobilePixor + scse(16) + msa(3&4 + mul + BN)`,`MobilePixor+scse(8) + msa(3 + concat + BN)`, and `MobilePixor + scse(8) + msa(3 + mul + BN)`, yield PTH file sizes of approximately 3.05 MB for the `scse(16)*` variants and 2.89 MB for the `scse(8)*` variants. Compared to `MobilePixor` (3.0 MB), this represents only a marginal increase.

In contrast, the student models demonstrate a substantial reduction in size, with PTH files of 0.45 MB, 0.50 MB, 0.45 MB, and 0.39 MB, respectively. These compact models are sufficiently lightweight to facilitate deployment on resource-constrained platforms, while still preserving accuracy within a stable range. The smaller footprint significantly reduces both model loading overhead and execution time, owing to the fewer trained weight matrices that must be processed.

Latency Analysis. Although the addition of scSE and MSA modules only moderately increases the number of parameters, these modules introduce additional operations and auxiliary layers that increase the computational depth of the network. Therefore, the observed inference latency does not scale linearly with the parameter count.

As summarized in Table 2, teacher backbone networks incorporating scSE and MSA generally exhibit higher latency than *MobilePixor*. In particular, the best-performing teacher variants reach approximately 129-132 ms, compared with 111.46 ms for *MobilePixor*. This confirms that the attention modules, especially scSE, introduce a non-negligible latency overhead on the tested embedded platform. Nevertheless, these models provide improved detection performance across multiple IoU thresholds, indicating an accuracy–latency trade-off.

By contrast, the distilled student networks in Table 3 achieve lower inference latency, typically within the range of 104–110 ms. Although the latency reduction is modest compared with the reduction in parameter count, the student models are substantially smaller and maintain competitive detection accuracy. This behavior suggests that inference time on the Radxa Rock 5C/RK3588S2 platform is affected not only by model size, but also by operator scheduling, memory access, computational depth, and backend optimization.

The measured latency corresponds to model-only inference and should not be interpreted as the full end-to-end latency of a robotic system. A complete agricultural robot pipeline would also include LiDAR acquisition, voxelization, localization, planning, control, and communication overhead. Therefore, the proposed student models are better described as compact perception models suitable for resource-constrained edge inference, while complete real-time robotic deployment requires further system-level validation.

5.4 Limitations and Future Work

Although the proposed framework achieves promising results, several limitations remain. First, the main quantitative evaluation is based on a Gazebo-generated synthetic dataset, where the training, validation, and testing subsets are split from the same mapped virtual plantation environment. Second, although augmentation was applied only after the original split and only to the training subset, all models were evaluated under a single fixed split. Repeated runs with different random seeds and statistical variation analysis were not conducted in the current study. Therefore, the reported improvements should be interpreted within this fixed-split experimental setting. Future work will include repeated-seed experiments, statistical significance analysis, and quantitative evaluation on annotated real rubber plantation datasets. Third, the reported latency corresponds only to model-level inference on the Radxa Rock 5C/RK3588S2 platform and does not include the complete robotic pipeline, such as LiDAR acquisition, voxelization, localization, planning, control, and communication. Therefore, full real-time robotic deployment requires further system-level validation.

6 Conclusion

An attention-augmented MobilePixor framework for LiDAR-based rubber tree detection has been presented. The incorporation of SCSE and selective MSA modules was shown to enhance spatial–semantic representations while maintaining efficiency. Through knowledge distillation, compact student networks were derived, achieving up to a 6.7× reduction in parameters while preserving accuracy close to that of the teacher backbone. The evaluation on a custom-built plantation dataset confirmed that robust detection performance can be maintained under significant architectural compression.

It was demonstrated that attention placement at shallow skip connections contributes most effectively to accuracy gains with minimal overhead. Furthermore, the distillation strategy was found to provide an effective mechanism for transferring representational capacity from the teacher to lightweight student variants, thereby demonstrating suitabil-

ity for low-latency inference on embedded platforms such as RK3588S2-based devices.

Future extensions are expected to include deployment on larger and more diverse field datasets, the integration of multi-modal sensor fusion (LiDAR–camera –mmWave radar), and optimization techniques such as quantization or hardware-aware compilation. These developments may further improve the practicality of the framework in real-world robotic applications.

Acknowledgements The authors gratefully acknowledge the support in terms of time and facilities provided by FPT University, Can Tho, Vietnam, which made this study possible.

Author Contributions Hoang N. Tran, Cong T. Nguyen, and Truong T. Duong developed the research objectives and core algorithms. Minh L. Nguyen, Nghi V. Nguyen, and Cong T. Nguyen drafted the manuscript, which Hoang N. Tran supervised and revised. All authors contributed to data analysis and interpretation, and approved the final version.

Funding This research received no external funding.

Data Availability The datasets generated and/or analyzed during the current study are available from the corresponding author upon reasonable request.

Declarations

Conflicts of interest The authors declare neither conflict of interest, nor competing interests.

Ethics Approval Not applicable (This study does not involve human participants, their data, or biological material).

Consent to Participate Not applicable (This article does not involve human participants).

Consent for Publication Not applicable (This article does not involve human participants).

References

1. Oliveira, L.F.P.; Moreira, A.P.; Silva, M.F.: Advances in forest robotics: A state-of-the-art survey. Robotics (2021). https://doi.org/10.3390/robotics10020053
2. Fang, J.; Shi, Y.; Cao, J.; Sun, Y.; Zhang, W.: Active navigation system for a rubber-tapping robot based on trunk detection. Remote Sens. (2023). https://doi.org/10.3390/rs15153717
3. Sowcharoensuk, C.: Industry Outlook 2023–2025: Rubber Processing. Krungsri Research, Aug. 22, 2023. [Online]. https://www.krungsri.com/en/research/industry/industry-outlook/agriculture/rubber/io/rubber-2023-2025, Accessed: Apr. 21, 2025.
4. Everstream Analytics. Natural rubber shortages intensify [Online]. Available: https://www.everstream.ai/risk-centers/natural-rubber-shortages-intensify/, Accessed: Apr. 20, 2025.
5. Grondin, V.; Fortin, J.-M.; Pomerleau, F.; Giguère, P.: Tree detection and diameter estimation based on deep learning. For. Int. J. For. Res. **96**(2), 264–276 (2023). https://doi.org/10.1093/forestry/cpac043
6. Shi, S.; Guo, C.; Jiang, L.; Wang, Z.; Shi, J.; Wang, X.; Li, H.: PV-RCNN: point-voxel feature set abstraction for 3D object detection. In: Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10529–10538 (2020). https://doi.org/10.1109/CVPR42600.2020.01054
7. Yang, B.; Luo, W.; Urtasun, R.: PIXOR: real-time 3D object detection from point clouds. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition, vol. 2018, pp. 7652–7660 (2018). https://doi.org/10.1109/CVPR.2018.00798
8. Silva, F.N.A., et al.: Resource-constrained onboard inference of 3D object detection and localisation in point clouds targeting self-driving applications. Sensors **21**(23), Art. no. 7933 (2021). https://doi.org/10.3390/s21237933
9. Wang, J.; Chen, X.; Cao, L.; An, F.; Chen, B.; Xue, L.; Yun, T.: Individual rubber tree segmentation based on ground-based LiDAR data and faster R-CNN of deep learning. Forests **10**(9), Art. no. 793 (2019). https://doi.org/10.3390/f10090793
10. Ren, S.; He, K.; Girshick, R.; Sun, J.: Faster R-CNN: towards real-time object detection with region proposal networks. IEEE Trans. Pattern Anal. Mach. Intell. **39**(6), 1137–1149 (2017). https://doi.org/10.1109/TPAMI.2016.2577031
11. Zhou, H.; Zhang, G.; Zhang, J.; Zhang, C.: Mapping of rubber forest growth models based on point cloud data. Remote Sens. (2023). https://doi.org/10.3390/rs15215083
12. Lang, A.H.; Vora, S.; Caesar, H.; Zhou, L.; Yang, J.; Beijbom, O.: "PointPillars: fast encoders for object detection from point clouds. In: Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12697–12705 (2019). https://doi.org/10.1109/CVPR.2019.01298.
13. Liu et al. Z.: Swin transformer: hierarchical vision transformer using shifted windows. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 9992–10002 (2021). https://doi.org/10.1109/ICCV48922.2021.00986
14. Zhou, Y.; Tuzel, O.: 'VoxelNet: end-to-end learning for point cloud based 3D object detection. In: Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4490–4499 (2018). https://doi.org/10.1109/CVPR.2018.00472
15. Drobnitzky, M.; Friederich, J.; Egger, B.; Zschech, P.: Survey and systematization of 3D object detection models and methods. Visual Comput. **40**(3), 1867–1913 (2024). https://doi.org/10.1007/s00371-023-02891-1
16. Nguyen, D.A.; Hoang, K.N.; Nguyen, N.T.; Nguyen, D.A.; Tran, H.N.: Enhancing indoor robot pedestrian detection using improved PIXOR backbone and Gaussian heatmap regression in 3D LiDAR point clouds. IEEE Access **12**, 9162–9176 (2024). https://doi.org/10.1109/ACCESS.2024.3351868
17. Martín-Martín, R.; Patel, M.; Rezatofighi, H.; Shenoi, A.; Gwak, J.; Frankel, E.; Sadeghian, A.; Savarese, S.: JRDB: a dataset and benchmark of egocentric robot visual perception of humans in built environments. IEEE Trans. Pattern Anal. Mach. Intell. **45**(6), 6748–6765 (2023). https://doi.org/10.1109/TPAMI.2021.3070543
18. Roy, A.G.; Navab, N.; Wachinger, C.: Recalibrating fully convolutional networks with spatial and channel "squeeze and excitation" blocks. IEEE Trans. Med. Imaging **38**(2), 540–549 (2019). https://doi.org/10.1109/TMI.2018.2867261
19. Preetha, R.; Jasmine Pemeena Priyadarsini, M.; Nisha, J.S.: Brain tumor segmentation using multi-scale attention U-Net with EfficientNetB4 encoder for enhanced MRI analysis. Sci. Rep. **15**(1), Art. no. 9914 (2025). https://doi.org/10.1038/s41598-025-94267-9
20. Hinton, G.; Vinyals, O.; Dean, J.: Distilling the knowledge in a neural network (2015). [Online]. arXiv preprint arXiv:1503.02531
21. Liu, T.; Chen, C.; Yang, X.; Tan, W.: Rethinking knowledge distillation with raw features for semantic segmentation. In: Proceedings of IEEE/CVF Winter Conference Applications of Computer Vision (WACV), Waikoloa, HI, USA, pp. 1155–1164 (2024). https://doi.org/10.1109/WACV57701.2024.00119

22. Law, H.; Deng, J.: CornerNet: detecting objects as paired keypoints. In: Proceedings of European Conference on Computer Vision (ECCV), pp. 734–750 (2018)
23. Tran, H.N.; Nguyen, N.N.N.; Le, N.Q.P.; Le, T.A.N.; Nguyen, A.D.: Grounding DINO and distillation-enhanced model for advanced traffic sign detection and classification in autonomous vehicles. Eng. Sci. Technol. Int. J. **64**, Art. no. 102028 (2025). https://doi.org/10.1016/j.jestch.2025.102028
24. Winter24: 3D PointCloud Rubber Tree Dataset. Hugging Face Datasets (2024). [Online]. https://huggingface.co/datasets/Winter24/3d_pointcloud_rubber_tree
25. Lin, T.Y.; Maire, M.; Belongie, S.; Hays, J.; Perona, P.; Ramanan, D.; Dollár, P.; Zitnick, C.L.: Microsoft COCO: common objects in context. In: Fleet, D.; Pajdla, T.; Schiele, B.; Tuytelaars, T. (Eds.) Computer Vision–ECCV 2014, pp. 740–755. Springer, Cham (2014)
26. Lagos, J.; Lempiö, U.; Rahtu, E.: FinnWoodlands dataset. In: Gade, R.; Felsberg, M.; Kämäräinen, J.-K. (Eds.) Image Analysis, pp. 95–110. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-31435-3_7

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.